

Nome: Mauro Nascimento Matias

RGM: 22723323

ESPECIFICAÇÃO ÁGIL
SISTEMA DE LOJA VIRTUAL DE CUPCAKES

Projeto: Cupcake Store

Tipo de sistema

Plataforma web responsiva para catálogo, pedidos demonstrativos, administração e controle de estoque.

Sumário

Apresentação . . . 4

1. Visão Geral do Negócio . . . 5

1.1 Delimitação do domínio . . . 5

2. Objetivos do Sistema . . . 6

2.1 Objetivo geral . . . 6

2.2 Objetivos específicos . . . 6

2.3 Qualidades técnicas . . . 6

3. Atores do Sistema . . . 7

4. Histórias de Usuário . . . 8

Tema: acesso e cadastro . . . 8

Tema: vitrine, carrinho e compra . . . 9

Tema: pedidos e logística demonstrativa . . . 10

Tema: administração . . . 11

5. Mapa de Afinidade . . . 14

6. Backlog Priorizado . . . 15

7. Tarefas Técnicas . . . 16

8. Diagrama de Casos de Uso . . . 18

8.1 Catálogo dos casos de uso . . . 20

9. Casos de Uso Expandidos . . . 21

UC-01 — Criar conta . . . 21

UC-02 — Autenticar usuário . . . 21

UC-03 — Consultar catálogo . . . 21

UC-04 — Buscar produto . . . 21

UC-05 — Consultar detalhes de produto . . . 22

UC-06 — Adicionar produto ao carrinho . . . 22

UC-07 — Alterar ou remover itens . . . 22

UC-08 — Finalizar pedido . . . 22

UC-09 — Consultar pedidos . . . 23

UC-10 — Cancelar pedido elegível . . . 23

UC-11 — Cadastrar produto . . . 23

UC-12 — Editar produto ou imagem . . . 24

UC-13 — Ajustar estoque . . . 24

UC-14 — Atualizar estado do pedido . . . 24

UC-15 — Consultar dashboard e relatórios . . . 24

10. Diagrama de Classes . . . 26

10.1 Principais relações . . . 26

11. Diagrama de Sequência — Login . . . 27

12. Diagrama de Sequência — Compra . . . 28

12.1 Fluxos de exceção . . . 28

12.2 Controle de concorrência . . . 29

13. Diagrama de Sequência — Entrega . . . 30

13.1 Cancelamento e reposição . . . 30

14. Wireframes . . . 32

14.1 Cobertura das telas . . . 35

15. Mapa Conceitual . . . 36

16. Mapa Navegacional . . . 37

17. Modelo Relacional PostgreSQL . . . 38

17.1 Dicionário resumido de dados . . . 39

17.2 Conceitos de integridade . . . 39

17.3 Notas de implementação . . . 39

18. Arquitetura e Tecnologias . . . 40

18.1 Visão geral . . . 40

18.2 Tecnologias observadas . . . 40

18.3 Organização observada . . . 40

18.4 Segurança e integridade . . . 41

18.5 Pagamentos e logística . . . 41

19. Testes e Evidências . . . 42

19.1 Inventário da suíte . . . 42

19.2 Matriz de cenários e testes relacionados . . . 42

20. Matriz de Rastreabilidade . . . 43

20.1 Operações da API . . . 43

21. Considerações Finais e Limitações . . . 44

22. Referências . . . 45

Documentação técnica oficial . . . 45



Apresentação

Este documento consolida a especificação acadêmica e a descrição técnica da Cupcake Store. A aplicação é um sistema web responsivo para demonstração de catálogo, carrinho, pedidos, administração de produtos e controle de estoque. Pagamento e entrega são exclusivamente demonstrativos: não há cobrança, liquidação financeira, frete ou operação logística real.

1. Visão Geral do Negócio

A Cupcake Store é o cenário acadêmico de uma confeitaria especializada em cupcakes artesanais. A operação simulada reúne um catálogo de sabores, consulta de disponibilidade, registro de pedidos e atividades administrativas. O sistema foi concebido para exercitar, em um domínio delimitado, a integração entre interface web, API e banco de dados relacional.

Em uma operação de pequeno porte, informações de produtos, pedidos e saldo podem ficar dispersas entre anotações ou ferramentas sem integração. Essa separação dificulta a consulta do catálogo, a conferência de disponibilidade e a recuperação do estado de uma compra. A aplicação demonstra como centralizar esses dados e explicitar regras de autorização e integridade.

O cliente navega pela vitrine, mantém um carrinho local, cria uma conta, escolhe uma forma de pagamento demonstrativa e acompanha o estado do pedido. O administrador mantém produtos e imagens, ajusta saldo com registro de movimentações, conduz pedidos pelos estados permitidos e consulta indicadores e relatórios derivados dos dados persistidos.

O produto é uma demonstração de Engenharia de Software, não uma loja comercial em operação. Não há transação financeira, cálculo de frete, prazo de entrega nem integração com transportadora. Os endereços servem para demonstrar o registro de dados e a preservação de um retrato histórico no pedido; não comprovam que uma entrega será executada.

1.1 Delimitação do domínio

O catálogo inicial contém oito sabores seedados: Chocolate Belga, Red Velvet, Baunilha, Morango, Limão Siciliano, Cenoura com Chocolate, Cookies & Cream e Doce de Leite. Os valores e saldos iniciais estão definidos no seed técnico e são dados demonstrativos, não pesquisa de preço de mercado.

O carrinho existe no navegador e não possui tabela própria. O pedido persistido conserva itens, nomes e preços do instante de checkout, bem como os dados de contato e endereço então associados ao cliente. O status acompanha apenas o estado atual, sem histórico de cada transição.

2. Objetivos do Sistema

2.1 Objetivo geral

Desenvolver e documentar uma aplicação web responsiva que demonstre os fluxos de uma loja virtual de cupcakes, integrando catálogo, cadastro, autenticação por sessão, checkout demonstrativo, acompanhamento de pedidos, gestão administrativa e persistência relacional.

2.2 Objetivos específicos

- Apresentar um catálogo consultável de produtos ativos, com descrição, imagem, preço e disponibilidade.
- Permitir que visitantes adicionem produtos ao carrinho sem autenticação e preservem o estado local durante navegação e login.
- Registrar clientes e autenticar os perfis de cliente e administrador por sessão persistida.
- Registrar pedidos com itens e preços históricos, validando no servidor os preços atuais e o estoque.
- Registrar uma modalidade de pagamento como informação demonstrativa, sem iniciar cobrança.
- Permitir consulta de pedidos e cancelamento enquanto o estado do pedido for elegível.
- Permitir ao administrador criar, alterar, ativar/desativar produtos e substituir imagens.
- Controlar saldos por ajustes assinados e movimentos de estoque vinculados ao domínio.
- Permitir avanço administrativo dos estados de pedido na ordem definida.
- Apresentar dashboard e relatórios calculados a partir dos pedidos e do catálogo.

2.3 Qualidades técnicas

Segurança, responsividade, rastreabilidade das operações, integridade de estoque, tratamento de erros e persistência são propriedades técnicas transversais. Elas não são funcionalidades autônomas nem substituem a validação de cada fluxo. As métricas apresentadas no painel descrevem registros do sistema e não resultados financeiros reais.

3. Atores do Sistema

Ator	Tipo	Responsabilidades e limites
Cliente	Pessoa autenticada com perfil **CLIENTE**	Consultar produtos; manter carrinho; criar conta; atualizar perfil; finalizar pedido demonstrativo; consultar os próprios pedidos; cancelar pedido próprio em preparo ou em transporte.
Administrador	Pessoa autenticada com perfil **ADMIN**	Manter produtos e imagens; ajustar estoque; consultar movimentações; consultar e atualizar pedidos; cancelar pedidos elegíveis; consultar dashboard e relatórios.

Visitantes podem consultar o catálogo e usar o carrinho local; para concluir um pedido precisam autenticar-se como cliente. O administrador é provisionado por variáveis de ambiente no seed e não participa do checkout de cliente.

O sistema executa validações, cálculo de totais e persistência, mas não é um ator humano. Não existe ator Entregador ou gateway de pagamento: a mudança de estado de entrega é manual pelo administrador e as modalidades de pagamento são apenas rótulos armazenados.

4. Histórias de Usuário

As estimativas e prioridades desta edição são reconstruções acadêmicas para ordenar o trabalho; não são evidência de planejamento histórico. A escala de prioridade é A (essencial), B (alta), C (média), D (baixa) e E (fora do MVP). Os pontos representam esforço relativo pela sequência de Fibonacci.

Tema: acesso e cadastro

HU-01 — Criar conta de cliente

Campo	Especificação
Requerente	Cliente visitante
Ação	Informar dados pessoais, contato, endereço e senha para abrir uma conta.
Comentários	O cadastro público cria perfil **CLIENTE** e inicia a sessão. Não há confirmação por e-mail.
História	Como visitante, quero criar uma conta de cliente para concluir pedidos e consultar minhas compras.
Critérios de aceitação	CA-01.1: o formulário valida os campos obrigatórios e o formato do endereço antes do envio. CA-01.2: o e-mail é normalizado e deve ser único; duplicidade retorna mensagem de campo. CA-01.3: cadastro válido cria perfil **CLIENTE**, sessão por cookie HttpOnly e mantém o carrinho ao retornar ao checkout.
Regras de negócio	RN-01.1: cadastro público nunca cria **ADMIN**. RN-01.2: senha com no mínimo 12 caracteres; endereço é informado pelo cliente sem consulta externa de CEP.
Requisitos não funcionais	RNF-01.1: senha armazenada como hash Argon2; respostas públicas não expõem hash ou token de sessão.
Prioridade	A
Pontos	5 — estimativa reconstruída

HU-02 — Autenticar e manter a conta

Campo	Especificação
Requerente	Cliente ou administrador
Ação	Entrar, consultar sessão, sair e atualizar dados próprios.
Comentários	O perfil define o redirecionamento e o acesso às áreas; a autorização é confirmada no backend.
História	Como usuário cadastrado, quero autenticar-me e administrar meus dados para usar somente as funções permitidas ao meu perfil.
Critérios de aceitação	CA-02.1: credenciais válidas criam sessão persistida e cookie HttpOnly; inválidas recebem mensagem genérica. CA-02.2: o usuário consulta e encerra a própria sessão, e o logout remove o cookie. CA-02.3: cliente altera nome, telefone, endereço e senha conforme as regras; troca de senha revoga sessões e exige novo login.
Regras de negócio	RN-02.1: papéis válidos são **CLIENTE** e **ADMIN**; cliente não acessa operações administrativas. RN-02.2: e-mail não é editável; endereço novo não altera o retrato já salvo em pedidos existentes.
Requisitos não funcionais	RNF-02.1: token opaco aleatório; somente o hash SHA-256 fica em **sessoes**, com prazo de validade e revogação.
Prioridade	A
Pontos	5 — estimativa reconstruída

Tema: vitrine, carrinho e compra

HU-03 — Consultar e localizar cupcakes

Campo	Especificação
Requerente	Visitante ou cliente
Ação	Navegar, pesquisar e abrir a página de um produto.
Comentários	Produto ativo sem saldo continua visível com indicação de esgotado.
História	Como visitante, quero consultar e pesquisar sabores para escolher um produto disponível.
Critérios de aceitação	CA-03.1: catálogo apresenta nome, imagem, descrição, preço e disponibilidade de produtos ativos. CA-03.2: busca por texto restringe resultados compatíveis e apresenta estado vazio quando nada corresponde. CA-03.3: detalhe de produto ativo apresenta descrição, preço e saldo; produto inexistente ou inativo não é oferecido como produto comprável.
Regras de negócio	RN-03.1: catálogo público não lista produtos inativos. RN-03.2: preço é exibido em BRL e nunca é calculado com ponto flutuante no frontend.
Requisitos não funcionais	RNF-03.1: interface adaptável a celular, tablet e desktop; imagens locais e alternativa visual padrão.
Prioridade	A
Pontos	8 — estimativa reconstruída

HU-04 — Manter carrinho local

Campo	Especificação
Requerente	Visitante ou cliente
Ação	Adicionar, ajustar e remover itens do carrinho.
Comentários	O carrinho não depende de conta e permanece após login ou logout.
História	Como comprador, quero revisar quantidades e subtotais antes de comprar para decidir o conteúdo do pedido.
Critérios de aceitação	CA-04.1: adicionar produto cria ou incrementa uma linha, e o subtotal reflete preço vezes quantidade. CA-04.2: alteração de quantidade e remoção atualizam total e indicador do carrinho. CA-04.3: ao abrir o carrinho, os produtos são reconciliados com a API; produto indisponível, saldo insuficiente ou preço alterado bloqueia checkout até revisão.
Regras de negócio	RN-04.1: apenas o carrinho fica no **localStorage**; credenciais e token nunca são armazenados ali. RN-04.2: preço novo exige aceite explícito; falha de API não significa produto removido.
Requisitos não funcionais	RNF-04.1: dados locais são validados defensivamente e a soma monetária utiliza centavos inteiros.
Prioridade	A
Pontos	5 — estimativa reconstruída

HU-05 — Finalizar pedido demonstrativo

Campo	Especificação
Requerente	Cliente autenticado
Ação	Confirmar itens e escolher uma modalidade demonstrativa.
Comentários	**PIX**, **CARTAO** e **DINHEIRO** são registros de escolha, sem cobrança. Não há campos de cartão ou integração financeira.
História	Como cliente, quero confirmar um pedido com os itens e o endereço do meu perfil para acompanhar a solicitação na demonstração.
Critérios de aceitação	CA-05.1: checkout requer cliente autenticado, carrinho não vazio, escolha de modalidade e preços/estoque revisados. CA-05.2: backend calcula total com os preços do banco, grava snapshots do pedido e registra pagamento demonstrativo. CA-05.3: repetição com mesma chave e conteúdo recupera o pedido sem novo débito; conteúdo divergente com a mesma chave é recusado. CA-05.4: indisponibilidade, preço alterado ou estoque insuficiente interrompem a operação sem confirmar débito parcial.
Regras de negócio	RN-05.1: o navegador não define usuário, total ou endereço do pedido. RN-05.2: estoque não pode ficar negativo e o checkout deve persistir pedido, itens, modalidade, baixa e movimentos de estoque de forma atômica.
Requisitos não funcionais	RNF-05.1: checkout usa chave UUID de idempotência, valores monetários **Decimal**/**NUMERIC(10,2)** e transação atômica. RNF-05.2: o contrato especifica serialização dos checkouts do mesmo cliente; a seção 12 distingue esse requisito do mecanismo implementado.
Prioridade	A
Pontos	13 — estimativa reconstruída

Tema: pedidos e logística demonstrativa

HU-06 — Consultar pedidos

Campo	Especificação
Requerente	Cliente autenticado
Ação	Consultar lista paginada e detalhe de pedido próprio.
Comentários	O detalhe exibe status atual, itens históricos e endereço preservado no checkout.
História	Como cliente, quero consultar minhas compras para saber o estado e os dados registrados em cada pedido.
Critérios de aceitação	CA-06.1: lista apresenta pedidos próprios do mais recente ao mais antigo, com paginação. CA-06.2: detalhe apresenta itens, valores, modalidade demonstrativa, endereço e status atual. CA-06.3: pedido inexistente ou pertencente a outro cliente retorna resposta não reveladora (404).
Regras de negócio	RN-06.1: alteração de perfil não modifica o snapshot do pedido. RN-06.2: só timestamps armazenados são exibidos; não se inventam datas de etapas intermediárias.
Requisitos não funcionais	RNF-06.1: respostas privadas usam **Cache-Control: no-store**; consultas respeitam autorização no servidor.
Prioridade	B
Pontos	5 — estimativa reconstruída

HU-07 — Cancelar pedido elegível

Campo	Especificação
Requerente	Cliente dono do pedido ou administrador
Ação	Solicitar cancelamento antes da conclusão.
Comentários	A modalidade demonstrativa permanece registrada; não há reembolso.
História	Como cliente, quero cancelar um pedido ainda em preparo ou transporte para encerrar a solicitação e restaurar o saldo reservado.
Critérios de aceitação	CA-07.1: pedido em **EM_PREPARACAO** ou **SAIU_PARA_ENTREGA** pode ser cancelado pelo dono; administrador pode cancelar pedido elegível. CA-07.2: cancelamento registra autor, data e motivo opcional, muda status e repõe cada item uma única vez. CA-07.3: pedido entregue, já cancelado ou de terceiro para cliente é recusado sem alteração do estoque.
Regras de negócio	RN-07.1: cancelamento é transacional e terminal; não existe estorno financeiro. RN-07.2: disputa com entrega ou outro cancelamento deve resultar em conflito e não em reposição duplicada.
Requisitos não funcionais	RNF-07.1: operação registra movimento de cancelamento associado a produto e pedido.
Prioridade	B
Pontos	5 — estimativa reconstruída

Tema: administração

HU-08 — Administrar produtos e imagens

Campo	Especificação
Requerente	Administrador
Ação	Cadastrar, editar, ativar/desativar e substituir imagem de produto.
Comentários	Remoção lógica preserva pedidos e referências históricas.
História	Como administrador, quero manter o catálogo atualizado para apresentar informações e imagens corretas aos clientes.
Critérios de aceitação	CA-08.1: cadastro valida nome, descrição, preço e saldo inicial, gera slug e cria movimento inicial quando há saldo. CA-08.2: edição altera campos permitidos, mas não saldo nem slug; produto pode ser desativado sem exclusão física. CA-08.3: upload aceita imagem JPG, PNG ou WebP dentro do limite e valida conteúdo real; falha não cria duplicação nem torna a referência atual inacessível.
Regras de negócio	RN-08.1: slug duplicado retorna conflito; não há DELETE físico. RN-08.2: imagens são armazenadas sob nome gerado pelo servidor e o caminho recebido não é controlado pelo cliente.
Requisitos não funcionais	RNF-08.1: upload de até 3 MiB com verificação de formato e acesso ao arquivo limitado à raiz de mídia.
Prioridade	B
Pontos	8 — estimativa reconstruída

HU-09 — Controlar estoque

Campo	Especificação
Requerente	Administrador
Ação	Registrar entrada ou saída e consultar movimentos.
Comentários	A quantidade é alterada por ajuste dedicado, checkout e cancelamento, não pela edição geral do produto.
História	Como administrador, quero ajustar e auditar o estoque para manter o saldo coerente com as operações simuladas.
Critérios de aceitação	CA-09.1: ajuste exige variação inteira diferente de zero e justificativa; saldo final negativo é rejeitado. CA-09.2: atualização do produto e movimento de ajuste são persistidos juntos. CA-09.3: histórico paginado apresenta tipo, variação, justificativa, pedido, autor e data quando disponíveis.
Regras de negócio	RN-09.1: entrada é positiva e saída negativa; usuário não edita ou apaga movimentos existentes. RN-09.2: cliente não pode ajustar saldo nem consultar a área administrativa.
Requisitos não funcionais	RNF-09.1: consistência transacional e trilha de movimentações associada ao produto.
Prioridade	B
Pontos	8 — estimativa reconstruída

HU-10 — Atualizar estado de pedido

Campo	Especificação
Requerente	Administrador
Ação	Avançar manualmente um pedido pelo fluxo demonstrativo.
Comentários	O aplicativo não modela entregador, geolocalização nem histórico completo de transições.
História	Como administrador, quero atualizar o estado do pedido na ordem definida para que o cliente veja sua situação atual.
Critérios de aceitação	CA-10.1: administrador pode avançar **EM_PREPARACAO** para **SAIU_PARA_ENTREGA** e depois para **ENTREGUE**. CA-10.2: a marca temporal de entrega é gravada ao concluir. CA-10.3: regressão, repetição inválida e avanço por cliente retornam erro sem alterar o pedido.
Regras de negócio	RN-10.1: **ENTREGUE** e **CANCELADO** são estados terminais; cancelamento usa operação própria. RN-10.2: atualização é manual e exclusivamente demonstrativa.
Requisitos não funcionais	RNF-10.1: transições são validadas no backend e serializadas por bloqueio do pedido no serviço atual.
Prioridade	B
Pontos	5 — estimativa reconstruída

HU-11 — Consultar indicadores e relatórios

Campo	Especificação
Requerente	Administrador
Ação	Consultar indicadores, séries temporais, ranking de produtos e registros por modalidade.
Comentários	Valores derivados de pedidos não equivalem a recebimentos; modalidades incluem pedidos cancelados conforme regra dos relatórios.
História	Como administrador, quero visualizar indicadores e relatórios do período para analisar a atividade registrada na demonstração.
Critérios de aceitação	CA-11.1: dashboard mostra contagens por status, pedidos entregues, valor de vendas concluídas, unidades, ticket médio e estado atual do estoque. CA-11.2: relatório de vendas e produtos considera pedidos atualmente entregues, agregados pela data de criação no período informado. CA-11.3: relatório de modalidades apresenta valores demonstrativos, incluindo registros cancelados, sem declarar que houve recebimento.
Regras de negócio	RN-11.1: vendas concluídas e ranking usam estado atual **ENTREGUE**; relatórios de modalidade incluem todos os status. RN-11.2: período é interpretado em **America/Sao_Paulo**; estoque do dashboard é atual e não depende da data consultada.
Requisitos não funcionais	RNF-11.1: soma monetária em decimal no servidor; períodos sem dados retornam zero válido, sem **NaN**.
Prioridade	C
Pontos	8 — estimativa reconstruída

5. Mapa de Afinidade

O agrupamento abaixo aproxima histórias que compartilham ator, dados ou fluxo. Cada história aparece uma única vez no mapa e no backlog.

Tema	Necessidade comum	Histórias
Acesso e cadastro	Identidade, sessão e dados do usuário	HU-01, HU-02
Vitrine e catálogo	Descoberta e consulta de produtos	HU-03
Carrinho e compra	Revisão local e registro do pedido	HU-04, HU-05
Pedidos e logística	Consulta, cancelamento e estado atual	HU-06, HU-07, HU-10
Gestão de produtos	Cadastro e apresentação do catálogo	HU-08
Estoque e movimentações	Saldo e trilha de ajustes	HU-09
Indicadores e relatórios	Síntese da atividade administrativa	HU-11

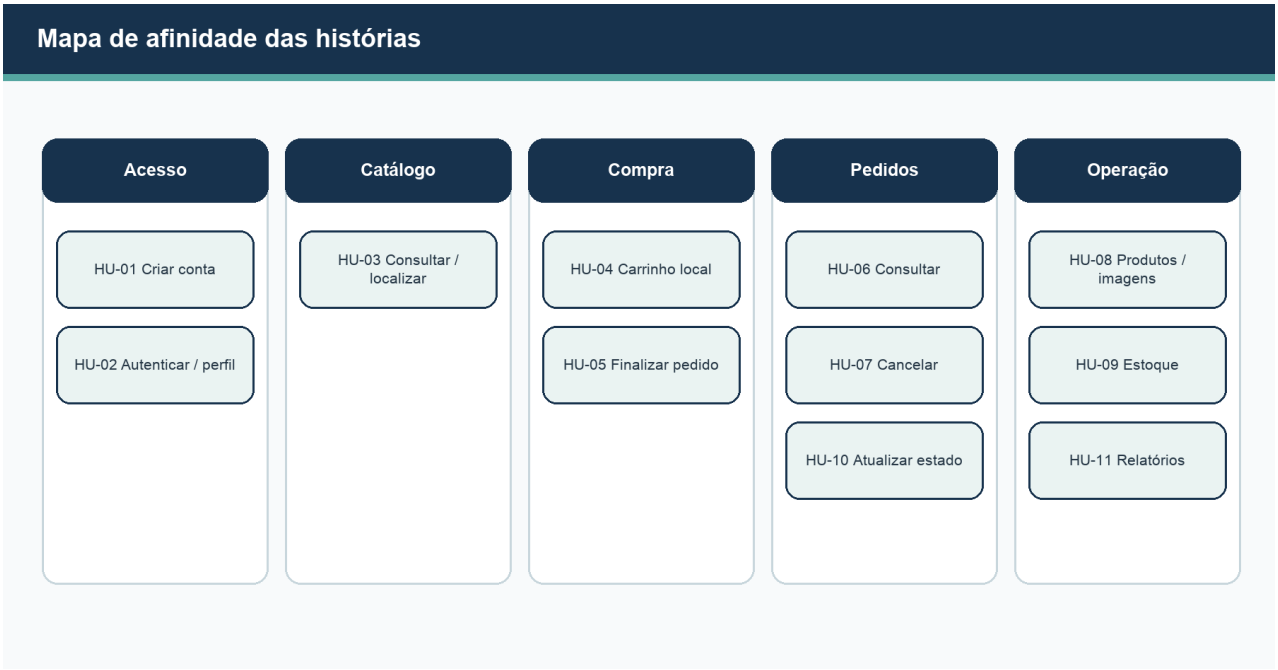


Figura 1 — Mapa de afinidade. Representação derivada das histórias desta edição.

6. Backlog Priorizado

A ordem prioriza a capacidade de completar o fluxo de compra e as dependências entre identidade, catálogo, carrinho e checkout. Itens administrativos dependem de produtos, pedidos e movimentos persistidos. A ordenação é uma proposta acadêmica reconstruída, não um registro de sprints concluídas.

Ordem	ID	História	Tema	Pontos	Prioridade
1	HU-01	Criar conta de cliente	Acesso e cadastro	5	A
2	HU-02	Autenticar e manter a conta	Acesso e cadastro	5	A
3	HU-03	Consultar e localizar cupcakes	Vitrine e catálogo	8	A
4	HU-04	Manter carrinho local	Carrinho e compra	5	A
5	HU-05	Finalizar pedido demonstrativo	Carrinho e compra	13	A
6	HU-06	Consultar pedidos	Pedidos e logística	5	B
7	HU-07	Cancelar pedido elegível	Pedidos e logística	5	B
8	HU-08	Administrar produtos e imagens	Gestão de produtos	8	B
9	HU-09	Controlar estoque	Estoque e movimentações	8	B
10	HU-10	Atualizar estado de pedido	Pedidos e logística	5	B
11	HU-11	Consultar indicadores e relatórios	Indicadores e relatórios	8	C
		Total estimado		75	

7. Tarefas Técnicas

As tarefas são unidades de trabalho técnicas vinculadas às histórias e aos módulos identificados no repositório. A tabela descreve entregáveis necessários à solução; não deve ser interpretada como histórico de execução.

ID	Histórias	Tarefa	Camada / módulo	Resultado esperado
TT-01	HU-01–HU-11	Configurar os serviços **frontend**, **backend** e **database**, variáveis e healthchecks.	Docker Compose	Inicialização reproduzível e serviços comunicáveis.
TT-02	HU-01, HU-02, HU-05–HU-11	Definir entidades, chaves, constraints e migration inicial.	SQLAlchemy / Alembic	Sete tabelas relacionais e integridade referencial.
TT-03	HU-03, HU-08	Preparar seed idempotente e assets locais de catálogo.	Seed / mídia	Oito sabores com imagens e alternativa local.
TT-04	HU-01, HU-02	Implementar cadastro, login, logout e consulta de sessão.	Controllers de autenticação	Sessões persistidas e cookie HttpOnly.
TT-05	HU-02	Implementar autorização por perfil, atualização de dados e troca de senha.	Usuários / dependências	Regras de acesso e revogação aplicadas no backend.
TT-06	HU-03	Implementar busca, listagem, detalhe e entrega segura de imagem.	API de produtos	Catálogo público limitado a produtos ativos.
TT-07	HU-04	Persistir e reconciliar carrinho no navegador.	Contexto e páginas de carrinho	Estado local recuperável e validado contra a API.
TT-08	HU-05	Validar preço, disponibilidade e chave de idempotência no checkout.	Serviço de checkout	Pedido sem total/endereço confiados ao navegador.
TT-09	HU-05, HU-07, HU-09	Persistir débito, reposição e movimentos de estoque.	Serviços de pedido e estoque	Operações atômicas e saldo não negativo.

ID	Histórias	Tarefa	Camada / módulo	Resultado esperado
TT-10	HU-06, HU-07, HU-10	Expor consulta, cancelamento e transição de estados.	API de pedidos	Autorização por titular/perfil e estados válidos.
TT-11	HU-08	Implementar manutenção de produto, slug e upload validado.	Administração de produtos	Catálogo atualizável sem apagar histórico.
TT-12	HU-09	Implementar ajuste assinado e histórico paginado.	Administração de estoque	Movimentos com tipo, saldo e justificativa.
TT-13	HU-11	Calcular dashboard e relatórios por período.	API de relatórios	Indicadores derivados do banco e regra temporal explícita.
TT-14	HU-01–HU-07	Implementar telas responsivas de catálogo, conta e compra.	Next.js / área cliente	Fluxo visitante–checkout–pedido.
TT-15	HU-08–HU-11	Implementar páginas administrativas e controle de acesso visual.	Next.js / área administrativa	Operações administrativas por perfil.
TT-16	HU-01–HU-11	Cobrir validação de API, sessão, pedidos, estoque e relatórios.	Pytest	Evidência automatizada de regras do backend.
TT-17	HU-03–HU-11	Cobrir navegação e fluxos no navegador em viewports móveis e desktop.	Playwright	Evidência ponta a ponta da interface integrada.
TT-18	HU-01–HU-11	Executar lint, tipagem, build e análise de falhas de integração.	Qualidade / CI local	Resultado documentado sem alegar execução inexistente.

8. Diagrama de Casos de Uso

Os diagramas usam atores UML fora da fronteira da **Cupcake Store** e elipses para os casos de uso. A visão geral agrupa as interações; as pranchas complementares detalham os 15 casos e suas associações, sem introduzir atores externos de pagamento ou entrega.



Figura 2 — Visão geral dos atores e grupos de casos de uso. Fonte editável: **diagramas/casos-de-uso.puml**.

UML — acesso, catálogo e carrinho

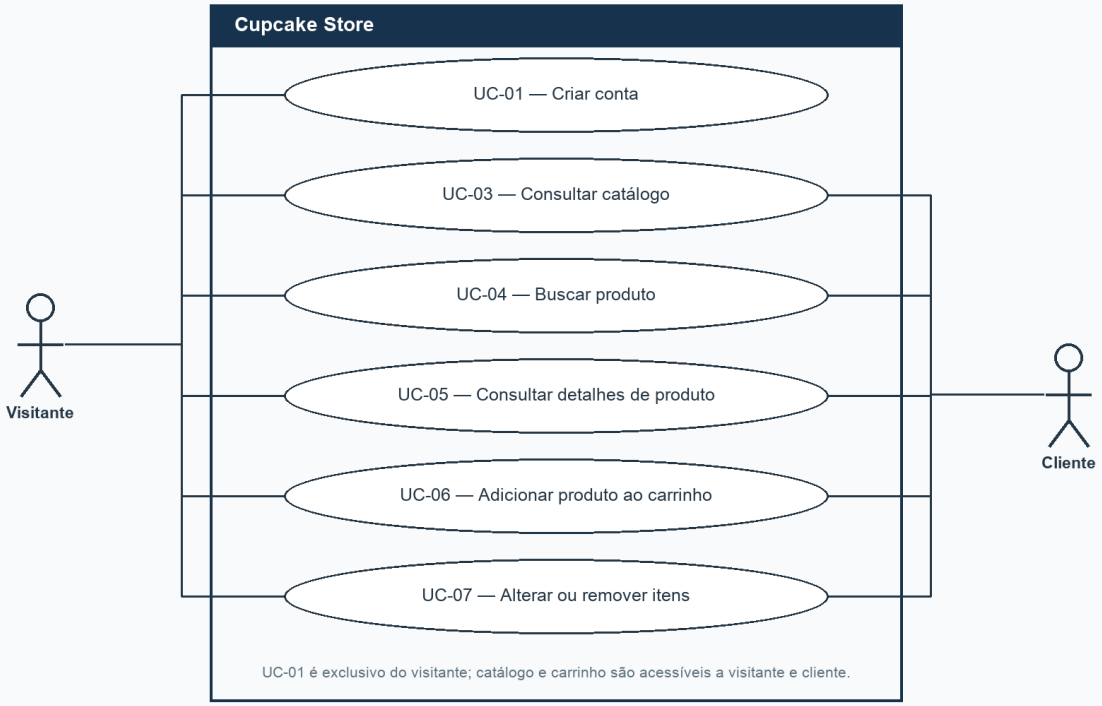


Figura 3 — Casos UC-01 e UC-03 a UC-07. Fonte editável: **diagramas/casos-de-uso.puml**.

UML — conta e pedidos

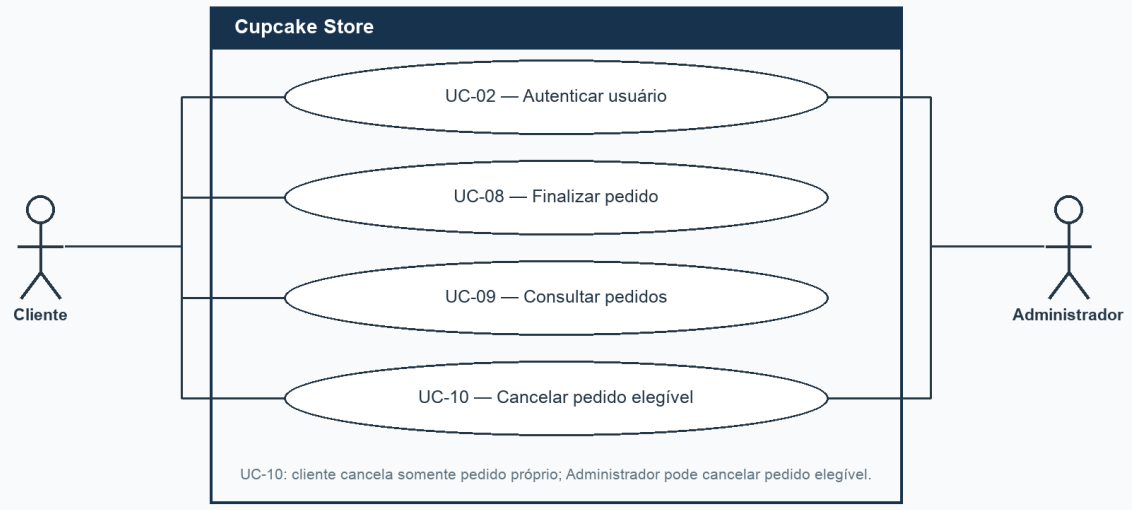


Figura 4 — Casos UC-02 e UC-08 a UC-10. Fonte editável: **diagramas/casos-de-uso.puml**.

UML — operações administrativas

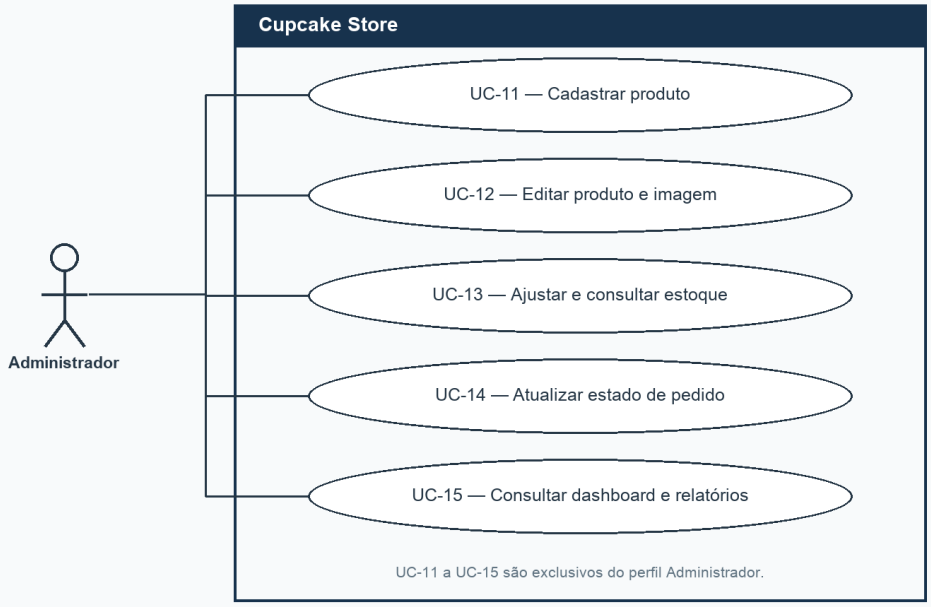


Figura 5 — Casos UC-11 a UC-15. Fonte editável: **diagramas/casos-de-uso.puml**.

8.1 Catálogo dos casos de uso

Caso	Nome	Ator principal	Histórias
UC-01	Criar conta	Cliente visitante	HU-01
UC-02	Autenticar usuário	Cliente / administrador	HU-02
UC-03	Consultar catálogo	Visitante / cliente	HU-03
UC-04	Buscar produto	Visitante / cliente	HU-03
UC-05	Consultar detalhes de produto	Visitante / cliente	HU-03
UC-06	Adicionar produto ao carrinho	Visitante / cliente	HU-04
UC-07	Alterar ou remover itens	Visitante / cliente	HU-04
UC-08	Finalizar pedido	Cliente	HU-05
UC-09	Consultar pedidos	Cliente	HU-06
UC-10	Cancelar pedido elegível	Cliente / administrador	HU-07
UC-11	Cadastrar produto	Administrador	HU-08
UC-12	Editar produto e imagem	Administrador	HU-08
UC-13	Ajustar e consultar estoque	Administrador	HU-09
UC-14	Atualizar estado de pedido	Administrador	HU-10
UC-15	Consultar dashboard e relatórios	Administrador	HU-11

9. Casos de Uso Expandidos

UC-01 — Criar conta

História relacionada: HU-01. Ator principal: cliente visitante. Descrição: registrar um cliente e iniciar sua sessão.

Pré-condições: visitante acessa cadastro; dados ainda não foram submetidos.

Pós-condições: usuário **CLIENTE** e sessão persistida; carrinho local preservado.

Fluxo básico: 1. Visitante abre cadastro. 2. Informa nome, e-mail, telefone, senha e endereço. 3. Interface valida os campos. 4. API normaliza o e-mail e verifica unicidade. 5. API grava usuário e sessão e retorna cookie HttpOnly. 6. Interface atualiza a autenticação e retoma o destino interno solicitado.

Alternativas e exceções: e-mail já cadastrado gera conflito associado ao campo; dados inválidos retornam validação; falha de rede mantém os valores preenchidos e informa que a operação não foi confirmada. Não existe confirmação por e-mail.

UC-02 — Autenticar usuário

História relacionada: HU-02. Ator principal: cliente ou administrador. Descrição: validar credenciais e abrir uma sessão.

Pré-condições: usuário cadastrado e ativo.

Pós-condições: sessão autenticada por cookie; usuário segue para uma rota interna permitida pelo perfil.

Fluxo básico: 1. Usuário informa e-mail e senha. 2. Frontend envia a solicitação à API same-origin. 3. Backend normaliza o e-mail e verifica a senha armazenada como hash. 4. Backend cria sessão e devolve cookie HttpOnly. 5. Frontend consulta o usuário autenticado. 6. Redireciona para a rota interna válida ou para a área padrão do perfil.

Alternativas e exceções: credenciais incorretas ou usuário inativo recebem resposta genérica; usuário sem sessão ao acessar recurso protegido é encaminhado ao login; destino externo ou não autorizado é descartado. O sistema não emite JWT.

UC-03 — Consultar catálogo

História relacionada: HU-03. Ator principal: visitante ou cliente. Descrição: visualizar produtos ativos e a disponibilidade atual.

Pré-condições: API disponível.

Pós-condições: catálogo exibido ou estado de erro recuperável.

Fluxo básico: 1. Ator abre a home ou a rota de produtos. 2. Frontend solicita página de produtos ativos. 3. API retorna itens ordenados e paginados. 4. Interface mostra imagem, nome, preço e indicação de estoque.

Alternativas e exceções: resposta vazia apresenta estado sem resultados; indisponibilidade da API apresenta erro e opção de tentar novamente; imagem ausente usa fallback local. Produtos inativos não aparecem no catálogo público.

UC-04 — Buscar produto

História relacionada: HU-03. Ator principal: visitante ou cliente. Descrição: localizar itens por texto compatível com nome ou descrição.

Pré-condições: catálogo acessível.

Pós-condições: lista filtrada ou estado sem correspondências.

Fluxo básico: 1. Ator informa o texto de busca. 2. Interface solicita os resultados com parâmetro **search**. 3. API filtra o catálogo público. 4. Interface atualiza os cards.

Alternativas e exceções: texto sem correspondência gera mensagem de busca vazia; falha de rede mantém consulta e permite repetir. Filtros por categoria não fazem parte do domínio atual.

UC-05 — Consultar detalhes de produto

História relacionada: HU-03. Ator principal: visitante ou cliente. Descrição: consultar informações e disponibilidade de um sabor específico.

Pré-condições: rota acessada por slug.

Pós-condições: detalhe do produto ativo exibido ou not-found.

Fluxo básico: 1. Ator seleciona um card. 2. Frontend consulta o detalhe pelo slug. 3. API retorna produto ativo, preço, descrição, imagem e quantidade. 4. Interface mostra opção de adicionar ao carrinho se houver saldo.

Alternativas e exceções: slug inexistente ou produto inativo resulta em 404 amigável; estoque zero mantém a página visível, mas desabilita a compra; falha de leitura disponibiliza nova tentativa.

UC-06 — Adicionar produto ao carrinho

História relacionada: HU-04. Ator principal: visitante ou cliente. Descrição: inserir produto no carrinho sem autenticação.

Pré-condições: produto ativo e quantidade disponível conhecida.

Pós-condições: carrinho local atualizado.

Fluxo básico: 1. Ator seleciona uma quantidade permitida. 2. Aciona “Adicionar ao carrinho”. 3. Frontend agrupa linha por produto, recalcula indicador e subtotais em centavos e persiste o carrinho local.

Alternativas e exceções: estoque zero impede inclusão; quantidade acima do saldo não pode ser selecionada; registro local corrompido é validado defensivamente e não deve quebrar a navegação.

UC-07 — Alterar ou remover itens

História relacionada: HU-04. Ator principal: visitante ou cliente. Descrição: revisar itens antes de iniciar checkout.

Pré-condições: carrinho aberto.

Pós-condições: linhas e total localizados ao estado revisado.

Fluxo básico: 1. Frontend consulta cada item na API. 2. Ator aumenta/diminui a quantidade ou remove item. 3. Interface atualiza subtotal, total e persistência local. 4. Checkout só é habilitado quando os itens foram reconciliados.

Alternativas e exceções: preço divergente mostra valor salvo e atual, exigindo aceite explícito; estoque menor exige ajuste; produto indisponível exige remoção; falha de API oferece nova tentativa e não é interpretada como exclusão.

UC-08 — Finalizar pedido

História relacionada: HU-05. Ator principal: cliente autenticado. Descrição: registrar um pedido com preços verificados e modalidade demonstrativa.

Pré-condições: perfil **CLIENTE**, carrinho não vazio, preço e estoque revisados, endereço no perfil e modalidade escolhida.

Pós-condições: pedido e itens persistidos; saldo debitado; movimento de venda e registro demonstrativo vinculados.

Fluxo básico: 1. API autentica o cliente e valida a chave UUID e o payload. 2. A operação inicia a transação. O contrato técnico exige bloqueio pessimista da linha do cliente antes da checagem de idempotência, mas esse bloqueio não foi identificado em **checkout_service.checkout**; o serviço consulta a chave sem bloquear o usuário. 3. A API calcula a impressão digital do payload e consulta o pedido pela combinação cliente/chave. 4. Para pedido novo, bloqueia os produtos em ordem crescente de ID e valida existência, atividade, preço e estoque. 5. Calcula o total com **Decimal** e copia para o pedido os dados atuais de contato/endereço. 6. Grava pedido, itens com snapshots, pagamento demonstrativo, baixa e movimentos **VENDA** na mesma transação. 7. Confirma com **COMMIT** e responde **201**; qualquer falha transacional causa **ROLLBACK**.

Alternativas e exceções: chave e conteúdo iguais retornam o pedido existente (**200**) sem novo débito; mesma chave com conteúdo diferente retorna **409 IDEMPOTENCY_CONFLICT**; preço alterado, produto indisponível e estoque insuficiente retornam conflitos específicos e não confirmam mutação parcial; timeout mantém carrinho e chave para repetição segura.

UC-09 — Consultar pedidos

História relacionada: HU-06. Ator principal: cliente autenticado. Descrição: listar e abrir pedidos pertencentes ao usuário.

Pré-condições: cliente com sessão válida.

Pós-condições: lista paginada ou detalhe próprio exibido.

Fluxo básico: 1. Cliente abre “Meus pedidos”. 2. API retorna lista recente limitada por **limit** e **offset**. 3. Cliente abre um registro. 4. API valida a titularidade e retorna itens e endereço históricos, modalidade, timestamps disponíveis e estado atual.

Alternativas e exceções: conta sem pedidos vê estado vazio; identificador inexistente ou alheio retorna 404; falha de consulta permite tentativa novamente.

UC-10 — Cancelar pedido elegível

História relacionada: HU-07. Ator principal: cliente dono do pedido ou administrador. Descrição: cancelar pedido antes do estado terminal de entrega.

Pré-condições: pedido em preparo ou transporte; cliente é titular ou ator é administrador.

Pós-condições: pedido **CANCELADO**, autor/data registrados e estoque reposto uma única vez.

Fluxo básico: 1. Ator abre detalhe e solicita cancelamento. 2. Interface solicita confirmação e aceita motivo opcional. 3. API bloqueia pedido e valida papel/titularidade/estado. 4. API bloqueia os produtos em ordem crescente, registra reposição e movimentos, atualiza cancelamento e confirma transação. 5. Interface atualiza o detalhe.

Alternativas e exceções: pedido entregue ou cancelado retorna conflito sem nova reposição; cliente alheio recebe 404; conflito com mudança simultânea de estado retorna **409 INVALID_STATUS_TRANSITION**; não há reembolso.

UC-11 — Cadastrar produto

História relacionada: HU-08. Ator principal: administrador. Descrição: criar um produto para o catálogo.

Pré-condições: sessão **ADMIN**.

Pós-condições: produto ativo criado e, se estoque inicial for positivo, movimento inicial registrado.

Fluxo básico: 1. Administrador informa nome, descrição, preço e saldo inicial. 2. API valida campos e deriva slug. 3. API grava produto com imagem padrão. 4. API registra estoque inicial positivo se aplicável. 5. Produto aparece no catálogo conforme estado ativo.

Alternativas e exceções: slug duplicado retorna conflito; payload inválido retorna erros de campo; acesso sem perfil ADMIN é recusado.

UC-12 — Editar produto ou imagem

História relacionada: HU-08. Ator principal: administrador. Descrição: atualizar dados permitidos ou imagem de produto.

Pré-condições: administrador autenticado e produto existente.

Pós-condições: informações alteradas e imagem anterior preservada até substituição confirmada.

Fluxo básico: 1. Administrador abre produto. 2. Altera nome, descrição, preço ou estado ativo, sem enviar saldo/slug. 3. API valida e persiste. 4. Para imagem, administrador envia arquivo e API valida conteúdo, limite e raiz segura. 5. Interface mostra produto atualizado.

Alternativas e exceções: produto inexistente retorna 404; arquivo inválido retorna erro de validação; falha de upload não recria produto nem deve deixar banco apontando para arquivo inexistente.

UC-13 — Ajustar estoque

História relacionada: HU-09. Ator principal: administrador. Descrição: registrar entrada ou saída manual de saldo.

Pré-condições: administrador autenticado; produto existente; variação inteira não nula e justificativa.

Pós-condições: saldo e movimento de ajuste persistidos.

Fluxo básico: 1. Administrador escolhe entrada ou saída e informa quantidade e justificativa. 2. Interface converte magnitude para variação assinada. 3. API bloqueia produto, calcula saldo e rejeita valor negativo. 4. API atualiza o produto e registra **AJUSTE** na mesma transação. 5. Interface atualiza saldo e histórico.

Alternativas e exceções: produto não encontrado retorna 404; saída acima do saldo retorna conflito sem mutação; cliente recebe acesso negado; timeout não é repetido automaticamente antes de consultar estado e histórico.

UC-14 — Atualizar estado do pedido

História relacionada: HU-10. Ator principal: administrador. Descrição: registrar avanço manual de preparo para entrega demonstrativa.

Pré-condições: administrador autenticado; pedido em estado que permite o próximo avanço.

Pós-condições: estado atualizado e timestamp de entrega gravado ao concluir.

Fluxo básico: 1. Administrador consulta pedido. 2. Solicita **SAIU_PARA_ENTREGA** ou **ENTREGUE** conforme o estado atual. 3. API bloqueia o pedido e valida transição. 4. API grava estado e data de entrega quando aplicável. 5. Cliente consulta novamente e observa o novo estado.

Alternativas e exceções: transição fora de ordem ou estado terminal retorna **409**; cliente não pode avançar pedido; não há geolocalização, evento de rastreamento ou data estimada.

UC-15 — Consultar dashboard e relatórios

História relacionada: HU-11. Ator principal: administrador. Descrição: consultar indicadores e agregados de um intervalo.

Pré-condições: sessão ADMIN; intervalo válido ou período padrão.

Pós-condições: indicadores e séries obtidos do banco.

Fluxo básico: 1. Administrador abre dashboard ou relatório. 2. Seleciona intervalo e granularidade quando aplicável. 3. API converte limites de data pelo fuso **America/Sao_Paulo**. 4. API agrega pedidos entregues para vendas/ranking e pagamentos registrados para o relatório demonstrativo. 5. Interface exibe totais, séries e

estados vazios.

Alternativas e exceções: datas parciais ou invertidas são rejeitadas; período sem dados retorna zeros; acesso sem perfil ADMIN é recusado. Valor demonstrativo de pagamento não equivale a dinheiro recebido.

10. Diagrama de Classes

O diagrama descreve as classes persistentes ORM observadas no módulo **backend/app/models/__init__.py**. Foram omitidos métodos genéricos e detalhes que não contribuem para a compreensão do domínio. As associações mostram as multiplicidades das chaves estrangeiras, incluindo os vínculos opcionais de cancelamento, pagamento e movimentação. O modelo relacional da seção 17 apresenta tipos, chaves e restrições físicas.

Diagrama UML de classes — associações e multiplicidades

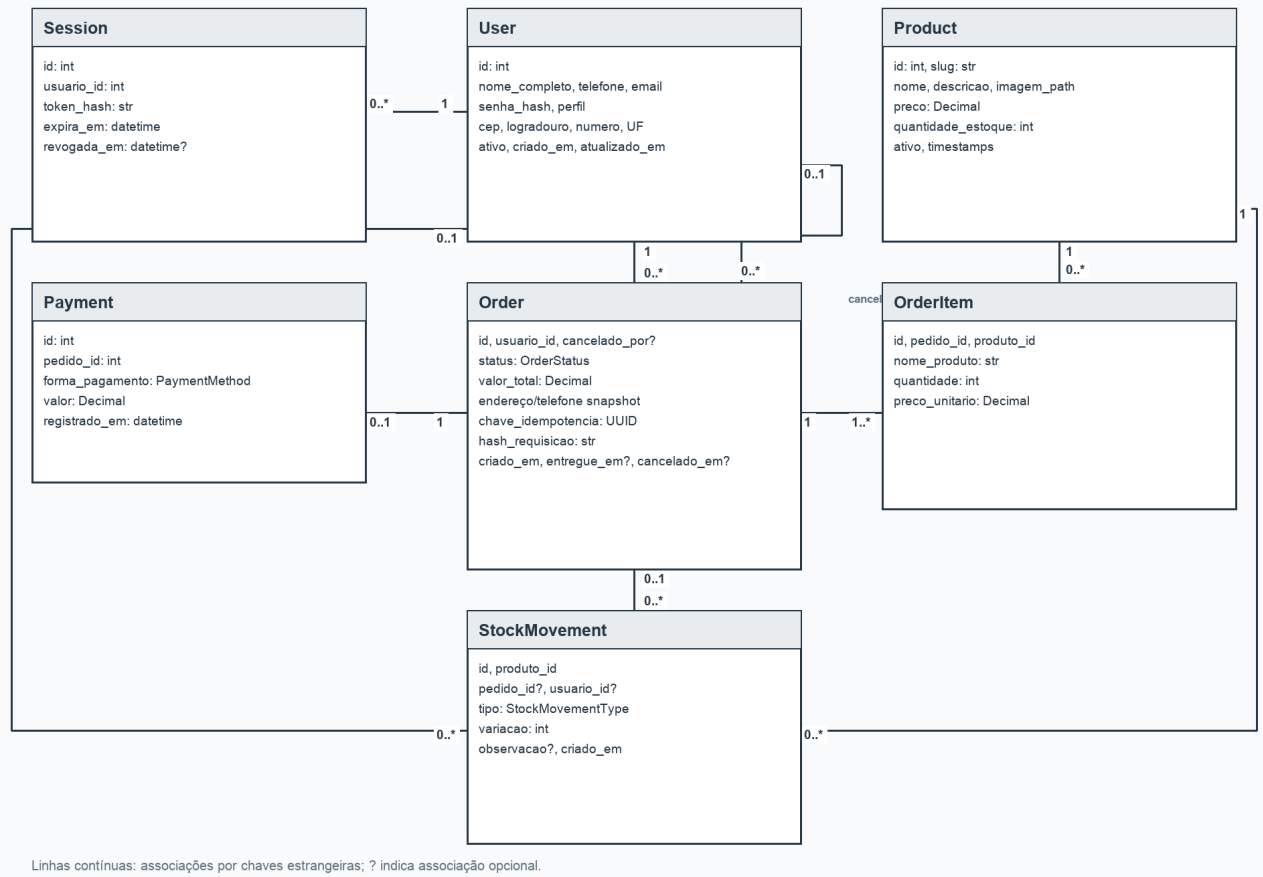


Figura 6 — Diagrama UML de classes. Os rótulos nas extremidades indicam as multiplicidades das associações.

10.1 Principais relações

- Um **User** possui **0..* Session** e **0..* Order**; cada sessão e pedido pertence a **1 User**. O cancelador é opcional (**0..1** por pedido), e cada usuário pode cancelar **0..*** pedidos.
- Um **Order** contém **1..* OrderItem**; cada item pertence a **1 Order** e referencia **1 Product**. Cada pedido pode ter **0..1 Payment**.
- Um **Product** relaciona-se a **0..* OrderItem** e **0..* StockMovement**. Cada movimentação referencia **1 Product** e, opcionalmente, **0..1 Order** e **0..1 User**; o carrinho permanece no navegador.

11. Diagrama de Sequência — Login

O fluxo abaixo resume autenticação por sessão opaca; não representa JWT. O cookie é HttpOnly e a tabela armazena somente o digest do token.

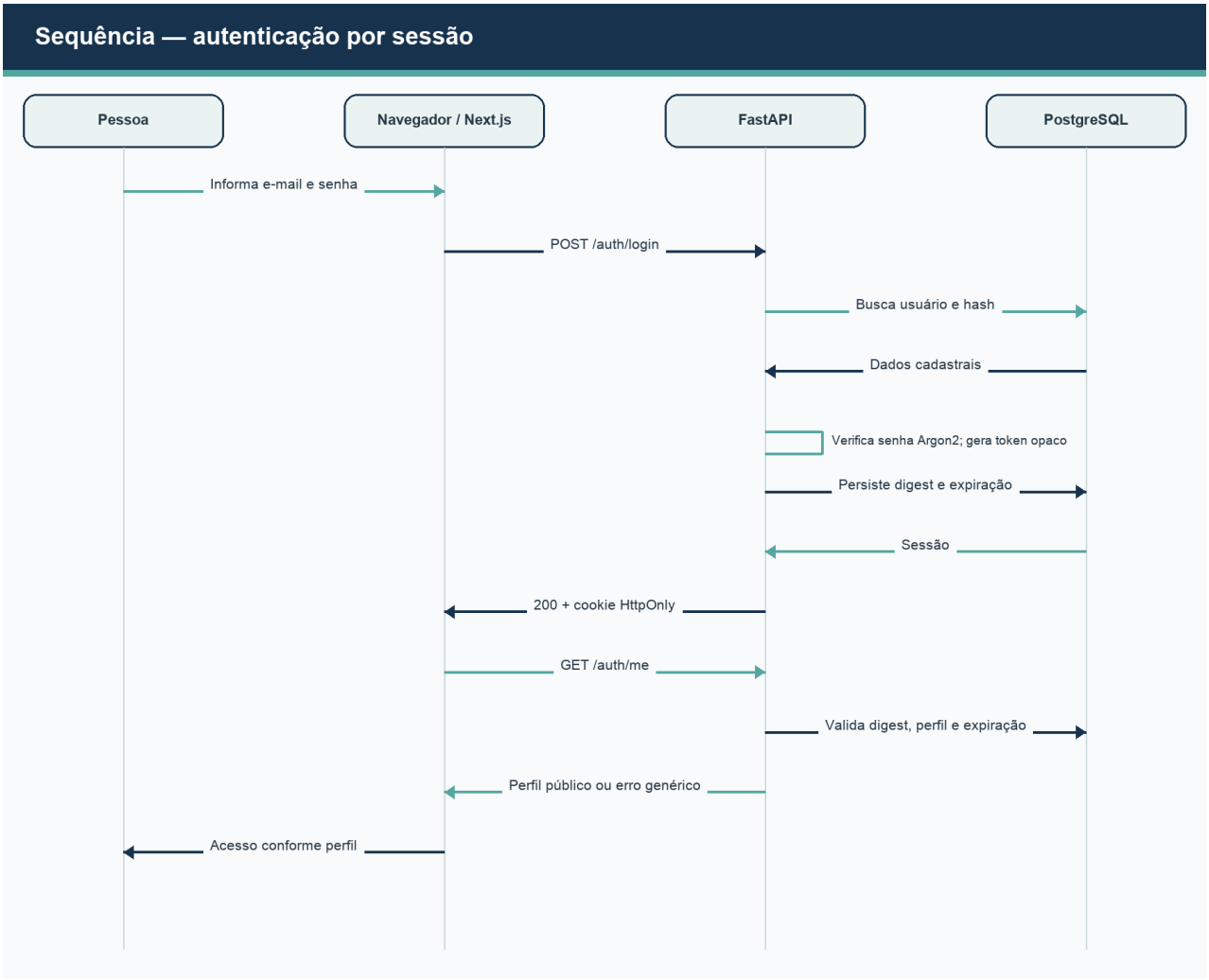


Figura 7 — Login, criação de sessão e consulta do usuário autenticado. Fonte editável: **diagramas/sequencia-login.mmd**.

Fluxo inválido: se o e-mail não corresponder a um usuário ativo ou a senha não validar, a API retorna erro genérico de credenciais inválidas e não cria sessão.

12. Diagrama de Sequência — Compra

O checkout começa com a autenticação do cliente e a validação do UUID e do payload. A sequência distingue os controles previstos no contrato técnico dos mecanismos observados na implementação. O total usa valores **Decimal**, e pedido, itens, pagamento demonstrativo, baixa e movimentos de estoque integram a mesma transação.

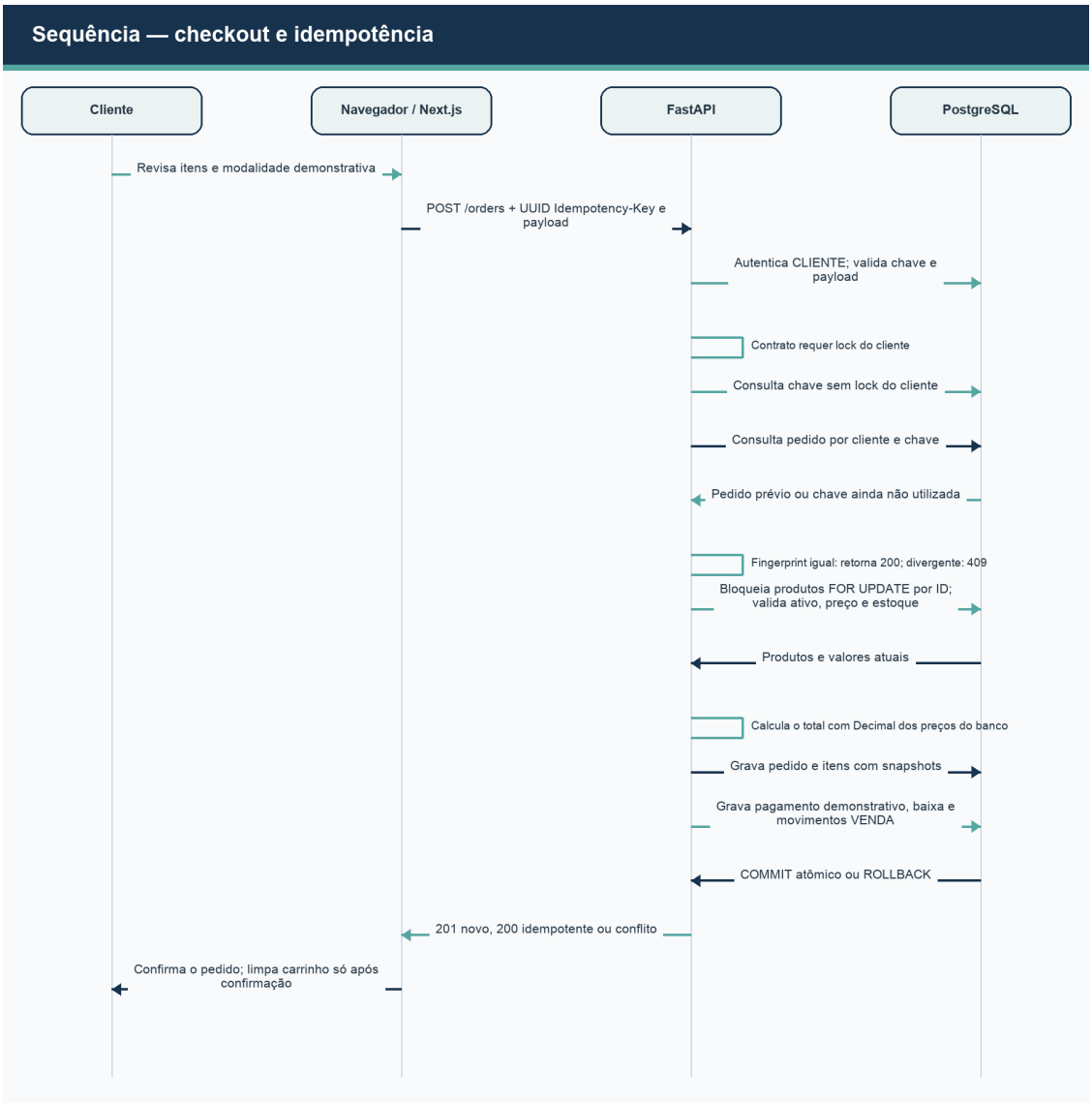


Figura 8 — Checkout, verificação de idempotência, bloqueio dos produtos e gravação atômica do pedido demonstrativo.

12.1 Fluxos de exceção

- Estoque insuficiente: API responde conflito com saldo disponível; nenhuma linha do pedido deve ser confirmada.
- Preço alterado: API informa produto, preço esperado e preço atual; o cliente deve revisar e aceitar explicitamente antes de uma nova tentativa lógica.
- Repetição idêntica: a chave e o conteúdo iguais recuperam o pedido registrado sem novo débito; conteúdo diferente associado à mesma chave é conflito.

- Timeout após envio: o carrinho não deve ser limpo antes de confirmação; a chave da tentativa é preservada para consulta/repetição idempotente.

12.2 Controle de concorrência

Contrato técnico: serializar checkouts simultâneos do mesmo cliente com bloqueio pessimista de sua linha antes da consulta de idempotência. Serviço observado: **checkout_service.checkout** consulta a impressão digital e a chave sem bloquear a linha do usuário; bloqueia somente as linhas dos produtos, em ordem crescente de ID. Assim, o bloqueio por cliente é requisito do contrato, não comportamento garantido pela implementação consultada.

13. Diagrama de Sequência — Entrega

Entrega é uma sequência de estados manual e demonstrativa. O cliente consulta o estado depois da alteração administrativa; o modelo guarda estado atual e timestamp de conclusão, não um histórico de cada etapa.

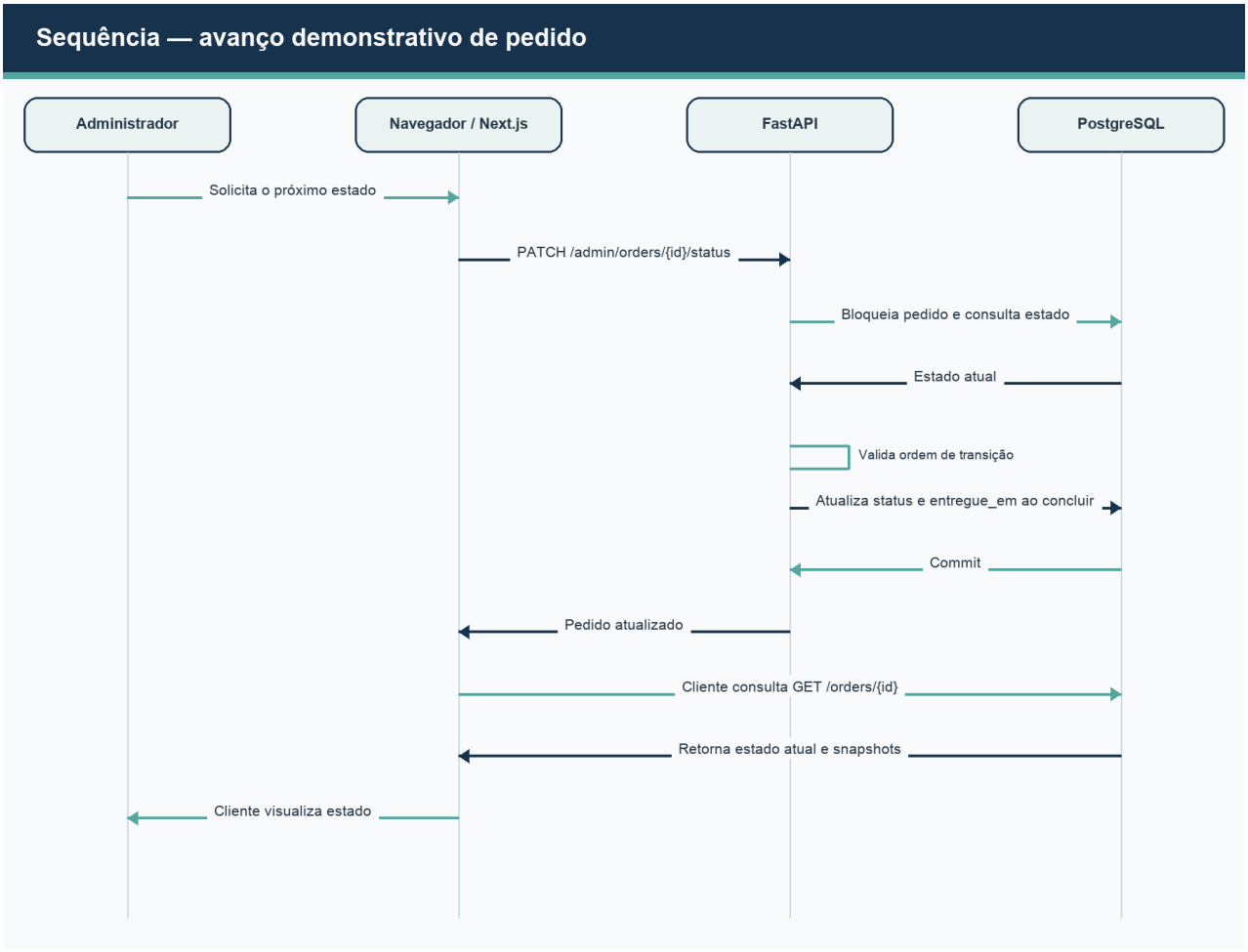


Figura 9 — Avanço administrativo de pedido até **ENTREGUE**.

13.1 Cancelamento e reposição

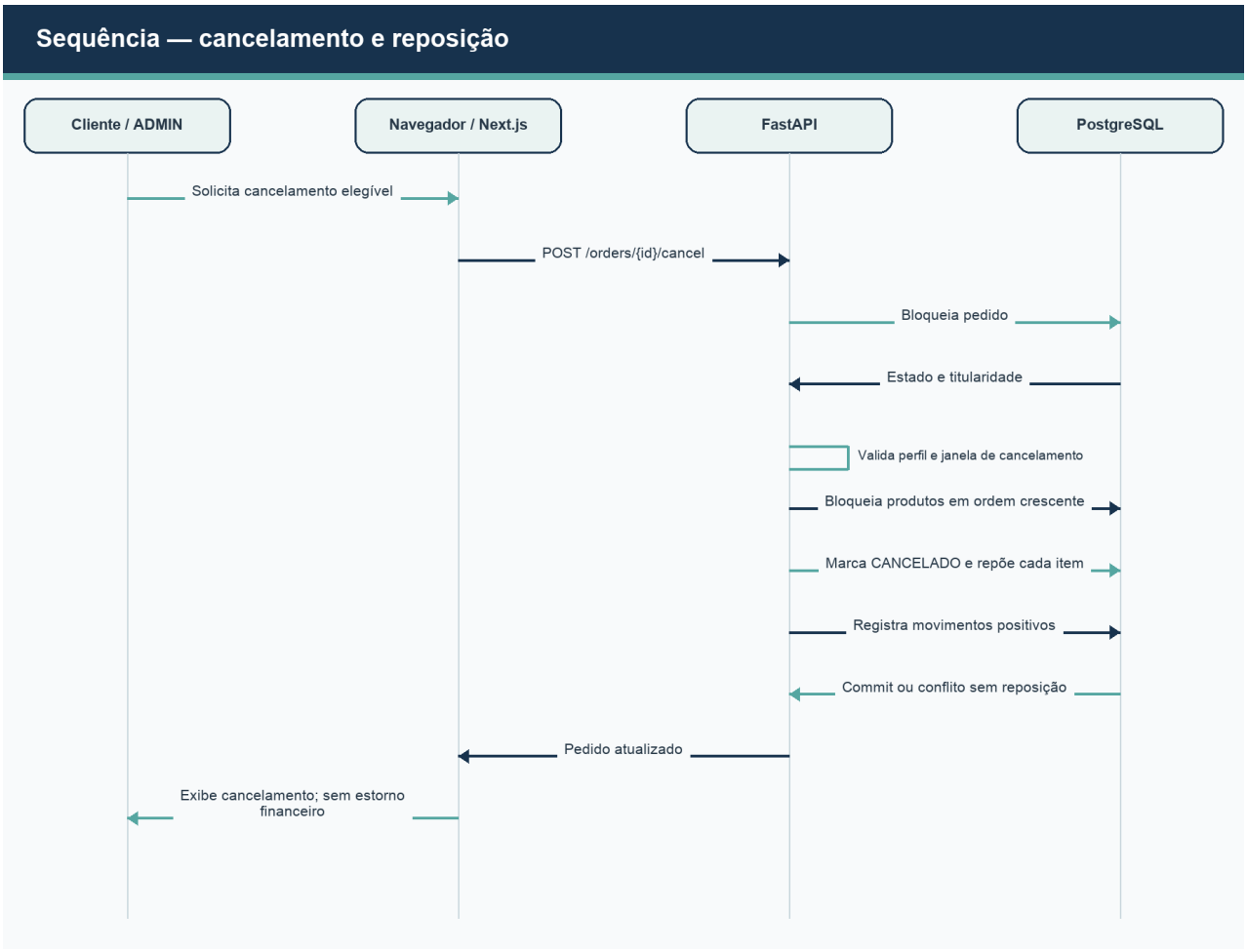


Figura 10 — Cancelamento elegível, autorização e movimento positivo de reposição. Fonte editável: **diagramas/sequencia-entrega-cancelamento.mmd**.

Cliente dono ou administrador pode cancelar pedido em preparo ou transporte. Administrador é o único perfil que avança o pedido. Entregue e cancelado são terminais; modalidade de pagamento permanece como registro demonstrativo e não há procedimento de estorno.

14. Wireframes

As figuras desta seção são wireframes conceituais, criados para comunicar hierarquia de conteúdo e ação. Não são capturas de tela do aplicativo nem evidências de execução. Os blocos neutros representam imagem, texto, formulário, tabela ou ação; cores e espaçamentos finais podem variar na implementação.

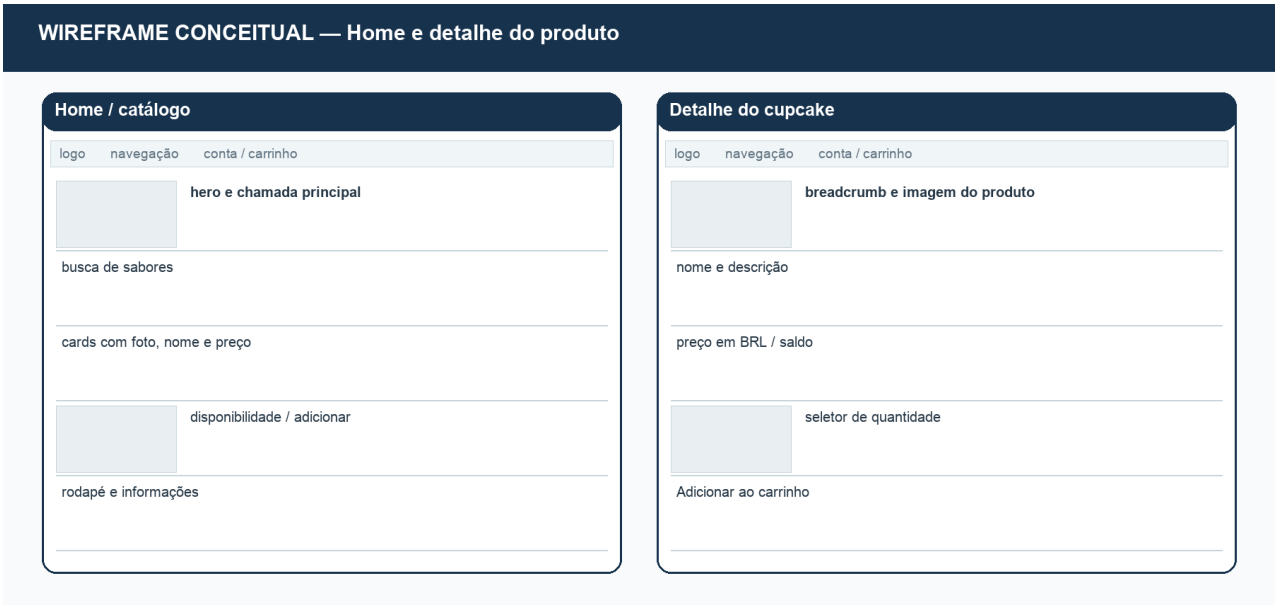


Figura 11 — Wireframes conceituais da home/catálogo e do detalhe do produto.

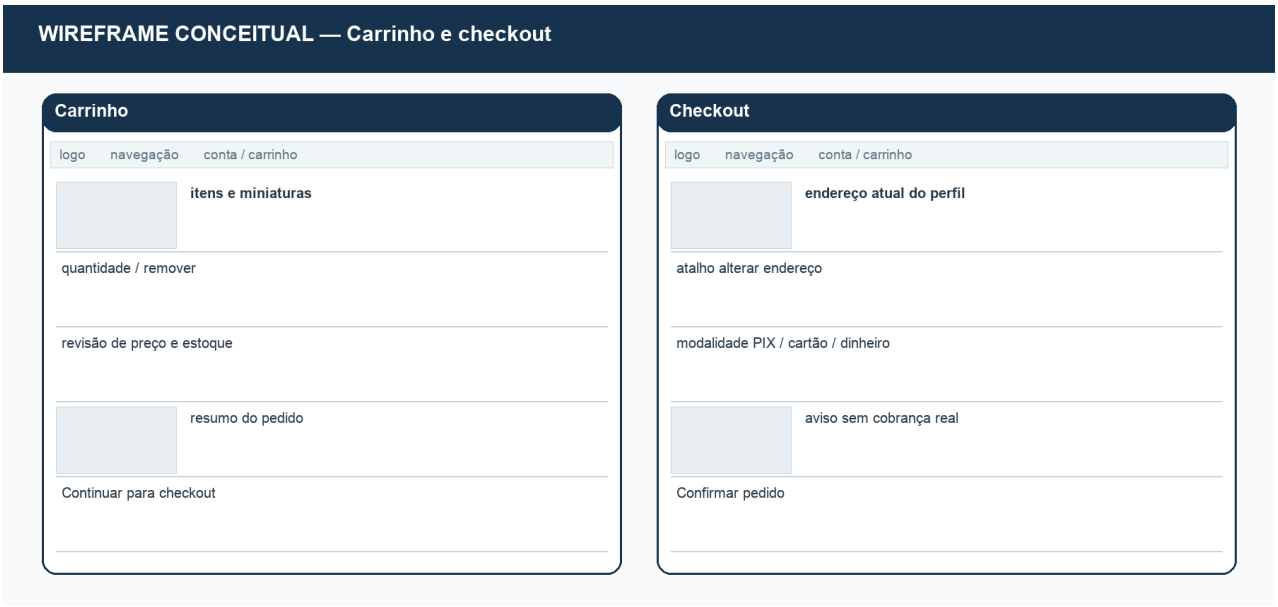


Figura 12 — Wireframes conceituais do carrinho e checkout, incluindo aviso de pagamento demonstrativo.



Figura 13 — Wireframes conceituais de login, cadastro e perfil.

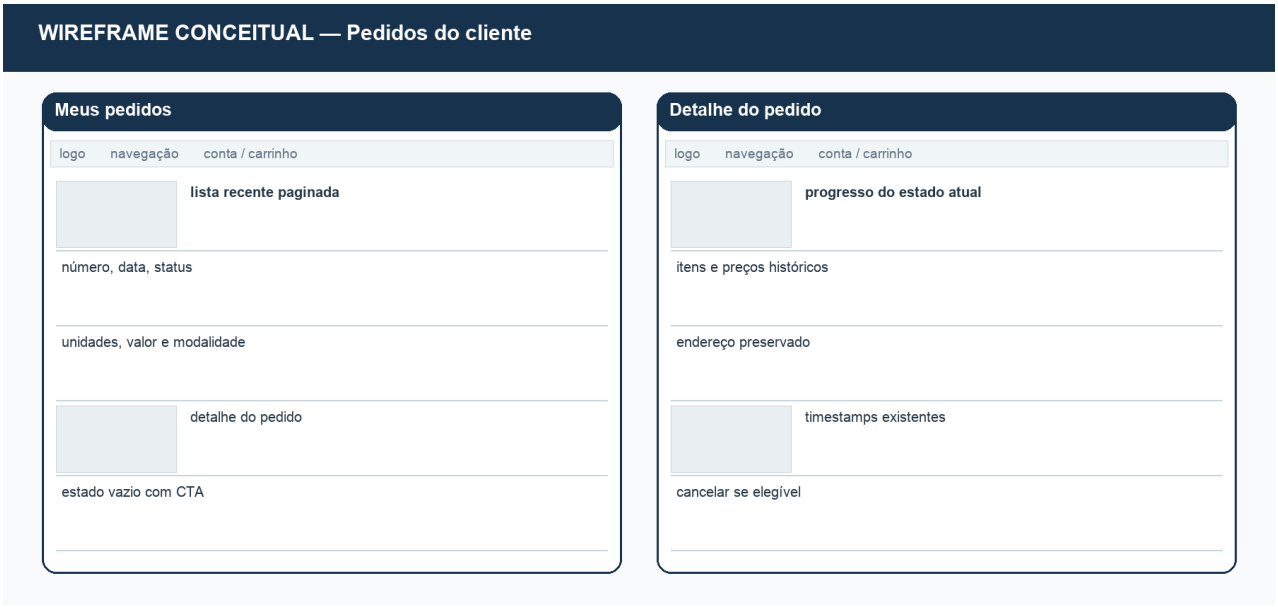


Figura 14 — Wireframes conceituais de pedidos do cliente e detalhe com estado atual.



Figura 15 — Wireframes conceituais do dashboard e da gestão de produtos.



Figura 16 — Wireframes conceituais de estoque e gestão de pedidos.

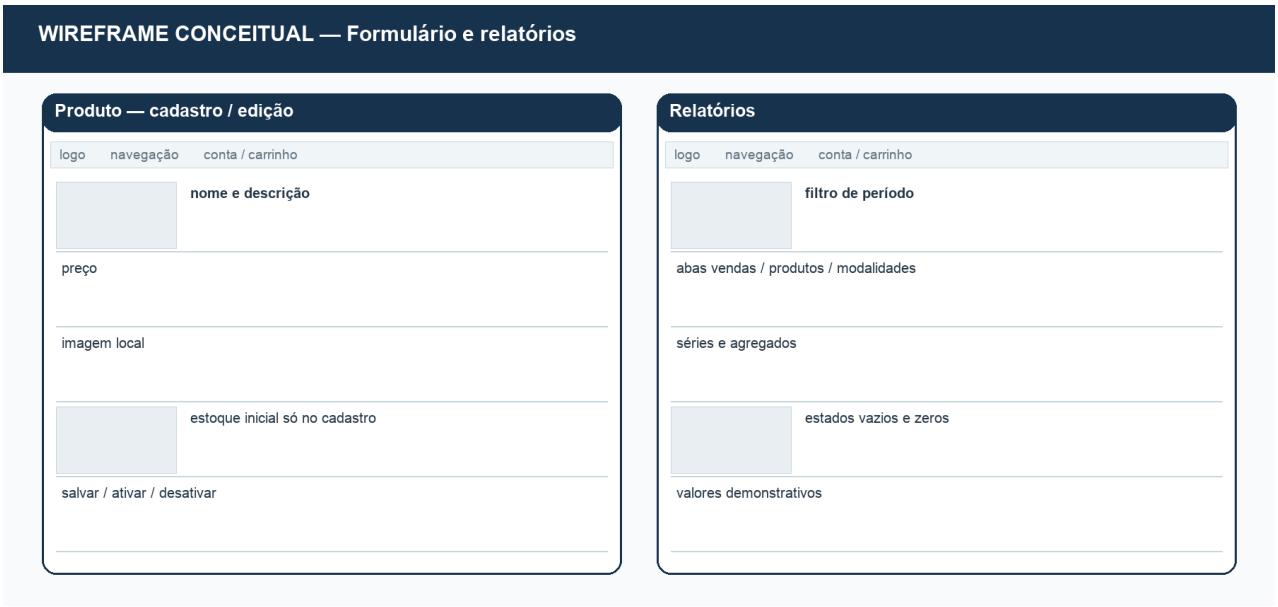


Figura 17 — Wireframes conceituais de cadastro/edição de produto e relatórios.

14.1 Cobertura das telas

Wireframe	Rotas representadas
Público	**/**, **/products/[slug]**
Compra	**/cart**, **/checkout**
Conta	**/login**, **/register**, **/account**
Pedidos	**/orders**, **/orders/[id]**
Administração — catálogo	**/admin**, **/admin/products**
Administração — operação	**/admin/stock**, **/admin/orders**, **/admin/orders/[id]**
Administração — formulário e dados	**/admin/products/new**, **/admin/products/[id]/edit**, **/admin/reports**

15. Mapa Conceitual

O mapa conceitual destaca as relações de domínio, sem substituir o diagrama de classes. O pedido é o ponto de integração entre usuário, itens, registro demonstrativo de modalidade e efeitos no estoque.



Figura 18 — Mapa conceitual do domínio e dos módulos funcionais.

16. Mapa Navegacional

O mapa apresenta as rotas do Next.js observadas no build e as condições de acesso por papel. Ele não representa endpoints REST, que usam o prefixo **/api/v1**.

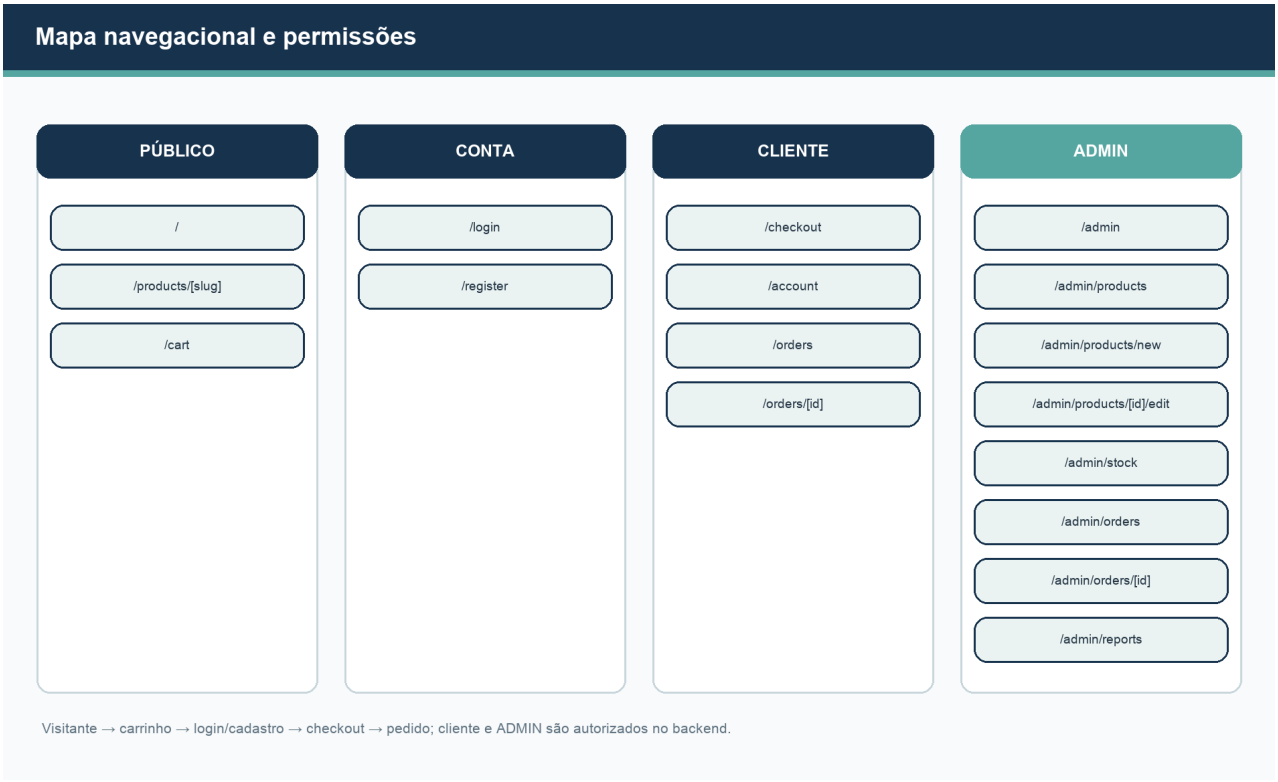


Figura 19 — Navegação pública, do cliente e administrativa. Fonte editável: **diagramas/mapa-navegacional.mmd**.

Área	Rotas	Acesso
Pública	**/**, **/products/[slug]**	Visitante ou usuário autenticado
Carrinho	**/cart**	Visitante ou usuário autenticado
Conta	**/login**, **/register**	Pública; usuário já autenticado é redirecionado conforme perfil
Cliente	**/checkout**, **/account**, **/orders**, **/orders/[id]**	Cliente autenticado; checkout também requer carrinho válido
Administração	**/admin**, **/admin/products**, **/admin/products/new**, **/admin/products/[id]/edit**, **/admin/stock**, **/admin/orders**, **/admin/orders/[id]**, **/admin/reports**	Administrador autenticado; API também valida perfil

Fluxos principais: visitante → catálogo → carrinho → login/cadastro → checkout → detalhe do pedido; cliente → pedidos → detalhe/cancelamento elegível; administrador → dashboard → produtos/estoque/pedidos/relatórios.

17. Modelo Relacional PostgreSQL

O modelo persistente reúne sete tabelas. A tabela de carrinho não existe: o carrinho é armazenado no navegador. O diagrama usa os nomes físicos declarados nos modelos/migration consultados. Pela regra do domínio, cada pedido confirmado contém pelo menos um item.

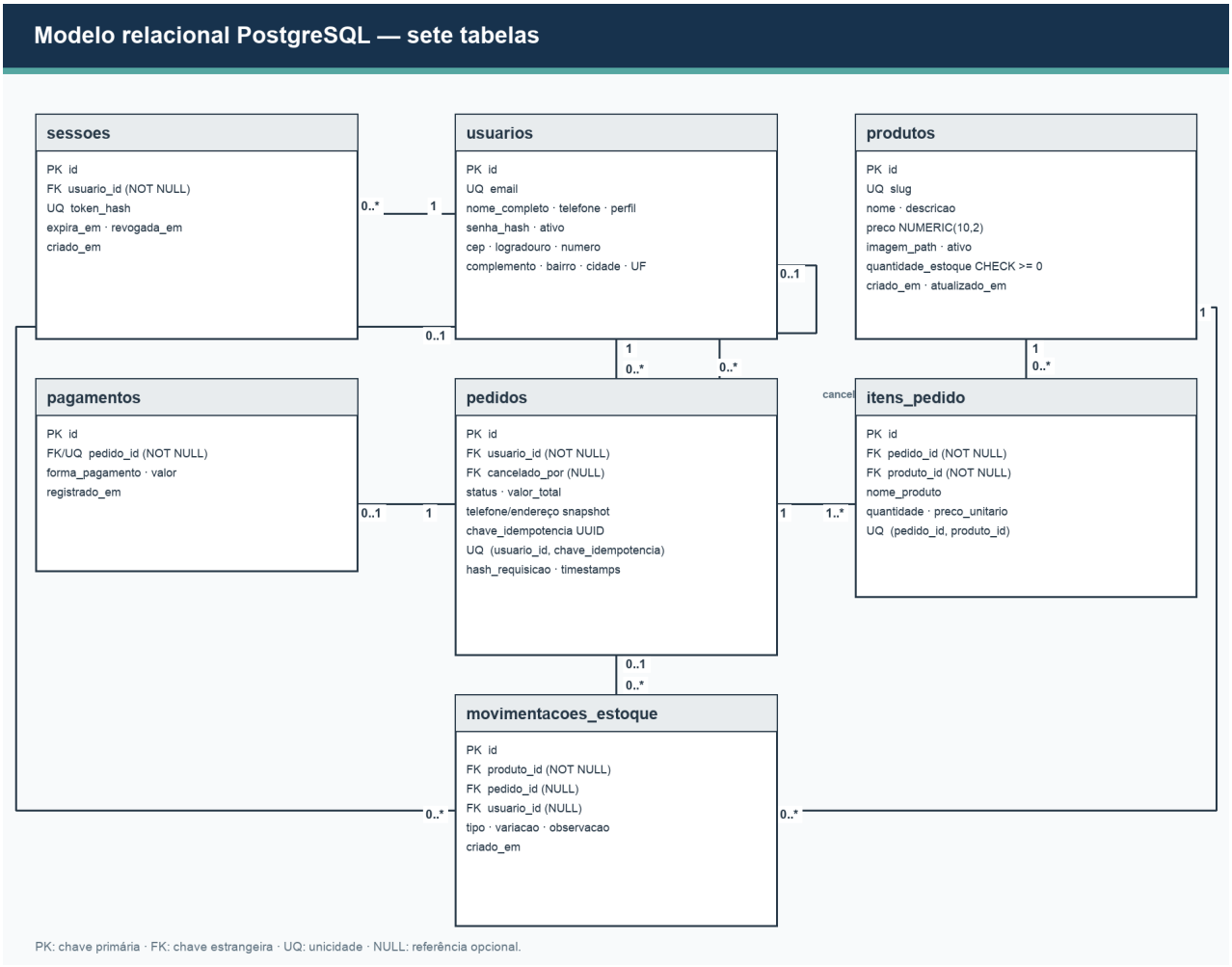


Figura 20 — Modelo entidade-relacionamento das sete tabelas. Fonte editável: **diagramas/entidade-relacionamento.mmd**.

17.1 Dicionário resumido de dados

Tabela	Chave e conteúdo principal	Integridade e função
usuarios	PK **id**; nome, telefone, e-mail, hash de senha, perfil, endereço atual, ativo e timestamps.	E-mail único; checks de perfil, telefone e CEP. Guarda dados da conta; **senha_hash** nunca é público.
sessoes	PK **id**; FK **usuario_id**; **token_hash**, expiração, revogação e criação.	Hash único e índice por usuário/expiração; permite sessão revogável sem armazenar token bruto.
produtos	PK **id**; slug, nome, descrição, preço, imagem, saldo, ativo e timestamps.	Slug único; preço positivo; saldo não negativo; índice de ordenação. Produtos são desativados logicamente.
pedidos	PK **id**; FK de cliente e cancelador; status, total, snapshot de contato/endereço, chave/hash idempotente e timestamps finais.	Restrição única cliente/chave; checks de status, total e coerência de timestamps; índices por cliente/data e status/data.
itens_pedido	PK **id**; FKs **pedido_id** e **produto_id**; nome, quantidade e preço do instante da compra.	Quantidade/preço positivos e produto único por pedido; preserva histórico independente do catálogo atual.
pagamentos	PK **id**; FK única **pedido_id**; modalidade, valor e data de registro.	Modalidade entre PIX/CARTAO/DINHEIRO e valor positivo; não representa autorização ou recebimento financeiro.
movimentacoes_estoque	PK **id**; FK produto e FKs opcionais pedido/usuário; tipo, variação, justificativa e data.	Variação diferente de zero; tipos controlados; índice por produto/data e unicidade parcial para venda/cancelamento do mesmo pedido/produto.

17.2 Conceitos de integridade

A chave primária identifica cada registro; as chaves estrangeiras mantêm referências entre registros relacionados. Restrições **CHECK** limitam valores válidos no banco e constraints **UNIQUE** evitam duplicidade em chaves de domínio. Índices apoiam filtros frequentes, como consulta de pedido por cliente e histórico de movimentações.

Os dados de endereço em **usuarios** representam o endereço atual. O pedido conserva um snapshot para que uma alteração futura de perfil não reescreva o endereço de uma compra antiga. De modo análogo, **itens_pedido** preserva nome e preço unitário do checkout.

O registro de pagamento armazena somente a modalidade declarada e o valor associado ao pedido. Não existem status bancário, transação de adquirente, cobrança ou estorno. O carrinho local evita persistência prematura de uma intenção de compra ainda não confirmada.

17.3 Notas de implementação

Os modelos e a migration inicial declaram as sete tabelas e seus relacionamentos. A migration usa BIGINT em PostgreSQL e sua variante INTEGER no SQLite empregado pelos fixtures automatizados.

18. Arquitetura e Tecnologias

18.1 Visão geral

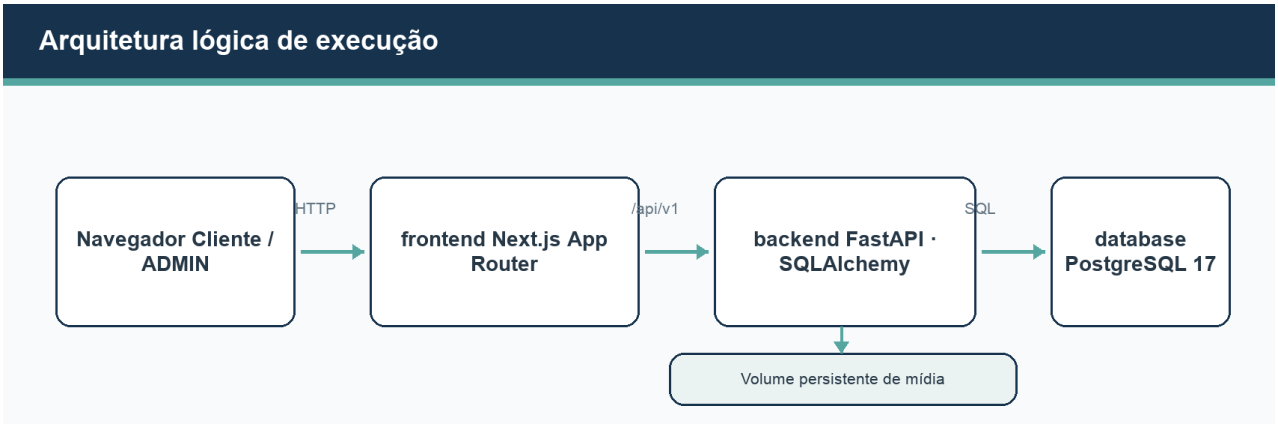


Figura 21 — Navegador, frontend Next.js, API FastAPI e banco PostgreSQL. O Compose declara os serviços **frontend**, **backend** e **database**.

O navegador acessa a aplicação Next.js. O frontend encaminha chamadas same-origin à API FastAPI sob **/api/v1**; o backend usa SQLAlchemy para persistir dados no PostgreSQL. Docker Compose define três serviços. O backend é publicado em loopback no compose atual e o banco não publica porta no host.

18.2 Tecnologias observadas

Camada	Tecnologia no repositório	Uso
Interface	Next.js 16.4.3, React 19.1.0, TypeScript, Tailwind CSS 4.1.11	Páginas App Router responsivas.
UI e validação	Radix UI, Lucide React, React Hook Form, Zod	Elementos acessíveis, ícones e validação de formulários.
Gráficos	Recharts 2.15.4	Gráficos de relatórios administrativos.
API	Python 3.12, FastAPI 0.162.0, Pydantic 2.10.6	Endpoints REST, schemas e validação.
Persistência	SQLAlchemy 2.0.55, Alembic 1.16.4, psycopg 3.3.2	ORM, migration explícita e driver PostgreSQL.
Banco/execução	PostgreSQL 17 no Compose; Uvicorn	Persistência relacional e servidor ASGI.
Testes	Pytest, HTTPX, Playwright 1.55.0	Testes de API e fluxos no navegador.

18.3 Organização observada

O backend mantém entidades em **app/models**, contratos em **app/schemas**, rotas em **app/controllers**, serviços de checkout/pedidos/estoque/seed em **app/services** e utilitários em **app/utils**. A aplicação registra **/health** fora dos 27 endpoints de negócio e inclui os controllers sob **/api/v1**.

O frontend usa rotas App Router, componentes, **lib** para API/carrinho/dinheiro e um provider compartilhado. O build enumera home, conta, catálogo, carrinho, checkout, pedidos e módulos administrativos. As telas dinâmicas de detalhe de produto e pedido são renderizadas sob demanda.

18.4 Segurança e integridade

O código examinado usa cookie de sessão HttpOnly, hash de senha Argon2 e digest SHA-256 do token em **sessoes**. Dependências de autenticação identificam usuário e papel; as rotas administrativas utilizam dependência de administrador. Middleware rejeita mutações com origem fora da lista configurada. Upload valida imagem e restringe leitura ao diretório de mídia.

O checkout calcula total com **Decimal**, verifica disponibilidade e preço, grava snapshots, cria registro demonstrativo e aplica movimentos de estoque na transação. As regras de idempotência e a distinção entre o contrato de concorrência e o mecanismo presente no serviço estão descritas na seção 12. Cancelamento e atualização de estado bloqueiam o pedido.

18.5 Pagamentos e logística

O checkout aceita identificadores PIX, cartão ou dinheiro apenas como modalidade informativa. Nenhum dado de cartão, CVV ou credencial bancária é solicitado. O relatório de pagamentos chama seus agregados de valores demonstrativos e inclui registros de diferentes estados. A aplicação não processa dinheiro nem prevê envio físico.

19. Testes e Evidências

19.1 Inventário da suíte

Camada	Evidência no repositório	Cobertura funcional declarada pela suíte
API/backend	**backend/tests/test_health.py**, **test_seed.py**, **test_api_flows.py**, **test_api_products.py**, **test_api_stock_users.py**, **test_api_orders_reports.py**	Health; seed idempotente; sessão, registro, logout, checkout/idempotência, estoque, produtos/imagens, perfil, estados, pedidos e relatórios.
Frontend/E2E	**frontend/tests/e2e/storefront.spec.ts** e **playwright.config.ts**	Viewports responsivos, fluxo cliente de compra/cancelamento e navegação administrativa. O teste administrativo ignora a execução sem credenciais de teste.
Qualidade frontend	scripts **lint**, **typecheck**, **build** em **frontend/package.json**	ESLint, TypeScript sem emissão e compilação de produção.

Os fixtures de backend usam SQLite em memória para os testes automatizados. O repositório também contém três cenários Playwright de ponta a ponta e scripts de qualidade do frontend.

19.2 Matriz de cenários e testes relacionados

ID	Funcionalidade	Cenário automatizado relacionado	Requisito ou comportamento relacionado
EV-01	Cadastro/sessão/logout	**test_registration_session_and_revocation**	Criação de conta, autenticação e revogação de sessão.
EV-02	Checkout idempotente/cancelamento	**test_checkout_is_idempotent_and_cancel_restores_stock_once**	Repetição idempotente e reposição única no cancelamento.
EV-03	Estoque sem débito parcial	**test_checkout_stock_conflict_does_not_partially_debit_products**	Conflito de estoque sem atualização parcial do pedido.
EV-04	Produtos/imagens	Testes de catálogo, upload, fallback e validação em **test_api_products.py**	Consulta e manutenção de produtos e imagens.
EV-05	Pedidos/relatórios	Testes de visibilidade, status, períodos e agregações em **test_api_orders_reports.py**	Autorização, transições e consultas agregadas.
EV-06	Catálogo responsivo	Primeiro cenário em **storefront.spec.ts**	Navegação do catálogo em diferentes viewports.
EV-07	Compra/cancelamento pelo cliente	Segundo cenário em **storefront.spec.ts**	Fluxo de compra e cancelamento demonstrativos.
EV-08	Área administrativa	Terceiro cenário em **storefront.spec.ts**	Navegação administrativa com credenciais de teste.

20. Matriz de Rastreabilidade

História	Casos de uso	Módulos/rotas observados	Testes relacionados
HU-01	UC-01	**controllers/auth.py**, **POST /api/v1/auth/register**	**test_api_flows.py**, **test_api_stock_users.py**
HU-02	UC-02	autenticação, **controllers/users.py**, **GET /api/v1/auth/me**, **PATCH /api/v1/users/me**, **PUT /api/v1/users/me/password**	**test_api_flows.py**, **test_api_stock_users.py**
HU-03	UC-03–UC-05	**controllers/products.py**, rotas públicas **/api/v1/products**	**test_api_products.py**, E2E catálogo
HU-04	UC-06–UC-07	**frontend/lib/cart.ts**, **frontend/components/store-provider.tsx**	E2E storefront; reconciliação no código frontend
HU-05	UC-08	**services/checkout_service.py**, **POST /api/v1/orders**	**test_api_flows.py**, testes de pedidos; E2E checkout
HU-06	UC-09	**controllers/orders.py**, **GET /api/v1/orders** e detalhe	**test_api_orders_reports.py**, E2E cliente
HU-07	UC-10	**services/order_service.py**, **POST /api/v1/orders/{id}/cancel**	**test_api_flows.py**, **test_api_orders_reports.py**, E2E cliente
HU-08	UC-11–UC-12	**controllers/products.py**, rotas **/api/v1/admin/products** e upload	**test_api_products.py**, E2E admin
HU-09	UC-13	**controllers/stock.py**, **services/stock_service.py**	**test_api_stock_users.py**, E2E admin
HU-10	UC-14	**services/order_service.py**, **PATCH /api/v1/admin/orders/{id}/status**	**test_api_orders_reports.py**, E2E admin
HU-11	UC-15	**controllers/reports.py**, dashboard e três relatórios administrativos	**test_api_orders_reports.py**, E2E admin

20.1 Operações da API

A aplicação registra 27 operações de negócio sob **/api/v1**, além do endpoint técnico **GET /health**. Os grupos são: autenticação/usuários (6), produtos/imagens (8), pedidos (7), estoque (2), dashboard/relatórios (4). A contagem é consistente com a afirmação estrutural testada no código; o quadro abaixo resume a cobertura funcional, não substitui o OpenAPI.

Grupo	Operações	Prefixo funcional
Autenticação e usuários	6	**/auth**, **/users**
Produtos e imagens	8	**/products**, **/admin/products**
Pedidos	7	**/orders**, **/admin/orders**
Estoque	2	**/admin/products/{id}/stock-***
Dashboard e relatórios	4	**/admin/dashboard**, **/admin/reports/***

21. Considerações Finais e Limitações

A Cupcake Store organiza em uma única solução demonstrativa os conceitos de requisitos, modelagem, interface web, autenticação, persistência e operações transacionais. A matriz de rastreabilidade liga histórias, casos de uso, módulos e testes relacionados.

O sistema não processa pagamentos reais, não emite comprovantes, não calcula frete e não coordena uma entrega física. O painel oferece métricas sobre registros do sistema, sem comprovar recebimento financeiro. O acompanhamento apresenta o estado atual do pedido, não histórico temporal detalhado.

Os diagramas apresentam modelos da estrutura e do comportamento descritos, e os wireframes são representações conceituais. A evolução para uso comercial exigiria decisões adicionais de segurança, operação, pagamentos e logística, fora do escopo atual.

22. Referências

Documentação técnica oficial

- Docker. Docker Compose: startup order. Disponível em: <https://docs.docker.com/compose/how-tos/startup-order/>. Acesso em: 9 nov. 2026.
- FastAPI. Documentação oficial. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 9 nov. 2026.
- PostgreSQL. Documentação oficial. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 9 nov. 2026.
- SQLAlchemy. SQLAlchemy 2.0 Documentation. Disponível em: <https://docs.sqlalchemy.org/en/20/>. Acesso em: 9 nov. 2026.
- Next.js. App Router e configuração de rewrites. Disponível em: <https://nextjs.org/docs/app/api-reference/config/next-config-js/rewrites>. Acesso em: 9 nov. 2026.
- OWASP. Cross-Site Request Forgery Prevention Cheat Sheet. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html>. Acesso em: 9 nov. 2026.
- Playwright. Documentação oficial. Disponível em: <https://playwright.dev/docs/intro>. Acesso em: 9 nov. 2026.